

Assembler–VM Interface Contract

Project: Stack-Based Virtual Machine

1 Purpose of This Contract

This document defines the **strict interface contract** between the **Assembler** and the **Virtual Machine (VM)**. The contract specifies how assembly programs are translated into bytecode and what guarantees the VM can rely on when executing that bytecode.

Key principle:

The assembler guarantees structural correctness of bytecode. The VM guarantees correct execution semantics.

2 Roles and Responsibilities

2.1 Assembler Responsibilities

The assembler is responsible for:

- Parsing assembly source files (`.asm`)
- Resolving labels into absolute byte offsets
- Encoding instructions into bytecode
- Rejecting invalid assembly programs

The assembler **must not**:

- Execute bytecode
- Simulate VM behavior
- Perform runtime validation

2.2 VM Responsibilities

The VM is responsible for:

- Loading bytecode files (`.bin`)
- Validating bytecode structure
- Executing bytecode semantics

The VM **must not**:

- Parse assembly syntax
- Resolve labels
- Guess instruction intent

3 Bytecode File Format

3.1 General Layout

The bytecode file is a raw binary stream consisting of:

- 1-byte opcode
- Optional 4-byte operand (little-endian)

No padding, alignment, or metadata is permitted.

3.2 Instruction Encoding Rules

- Opcode size: **1 byte**
- Operand size (if present): **4 bytes**
- Operand byte order: **little-endian**

4 Instruction Set Specification

4.1 Stack and Data Movement

Mnemonic	Opcode	Operand	Description
PUSH	0x01	int32	Push literal onto stack
POP	0x02	—	Pop top of stack
DUP	0x03	—	Duplicate top of stack
HALT	0xFF	—	Stop execution

4.2 Arithmetic and Logical Operations

Mnemonic	Opcode	Stack Effect
ADD	0x10	$[a, b] \rightarrow [a + b]$
SUB	0x11	$[a, b] \rightarrow [a - b]$
MUL	0x12	$[a, b] \rightarrow [a \times b]$
DIV	0x13	$[a, b] \rightarrow [a / b]$
CMP	0x14	$[a, b] \rightarrow [a < b]$

4.3 Control Flow

Mnemonic	Opcode	Description
JMP	0x20	Unconditional jump
JZ	0x21	Jump if top of stack is zero
JNZ	0x22	Jump if top of stack is non-zero

4.4 Memory Operations

Mnemonic	Opcode	Description
STORE	0x30	Store stack value into memory[index]
LOAD	0x31	Load memory[index] onto stack

4.5 Function Calls

Mnemonic	Opcode	Description
CALL	0x40	Call function at address
RET	0x41	Return from function

5 Assembler Guarantees

For any bytecode file produced by the assembler, the following are guaranteed:

- All opcodes are valid
- All required operands are present
- No instruction is truncated
- All labels are resolved to valid byte offsets
- No undefined labels exist
- Instruction boundaries are preserved

6 VM Assumptions

The VM may safely assume:

- Program Counter (PC) always points to a valid instruction
- Operand bytes exist when required
- Jump addresses are valid instruction boundaries
- No illegal opcode appears at runtime

7 VM Validation Rules

Before execution, the VM must validate:

- Opcode validity
- Instruction truncation
- Presence of at least one HALT instruction

If validation fails, the VM must:

- Print an error message to `stderr`
- Terminate execution with a non-zero exit code

8 Error Handling Boundary

- Syntax errors are assembler errors
- Structural bytecode errors are VM errors
- Runtime semantic errors are VM errors

9 Ownership Rules

- The VM owns the bytecode buffer after loading
- The assembler never accesses runtime memory
- The VM does not modify bytecode contents

10 Non-Goals

The following are explicitly outside the scope of this contract:

- Bytecode optimization
- Debug symbols
- Source-level error recovery
- Self-modifying code

11 Design Invariant

Assembler encodes intent. VM enforces semantics.

12 Conclusion

This contract establishes a clean and verifiable boundary between assembly and execution. Any system conforming to this contract is guaranteed to be modular, testable, and robust.

13 VM Input Specification

13.1 Overview

The Virtual Machine (VM) receives input exclusively in the form of a **compiled bytecode file**. No textual assembly input is ever processed by the VM.

Design invariant:

The VM only understands bytes, never text.

13.2 Invocation Interface

The VM is invoked from the command line using the following syntax:

```
vm <bytecode_file>
```

Exactly one argument must be supplied.

13.3 Command-Line Argument Rules

- The argument must be a path to a bytecode file
- No additional arguments are permitted
- Missing or extra arguments are treated as errors

13.4 Input File Type

The input file must satisfy all of the following:

- Binary file generated by the assembler
- Contains raw bytecode instructions
- No textual data or assembly syntax

13.5 Input Loading Process

Upon invocation, the VM performs the following steps:

1. Open the bytecode file in binary mode
2. Read the entire file into memory
3. Allocate a byte buffer to store the program
4. Initialize the VM state using the loaded bytecode

13.6 Ownership of Input Data

Once the bytecode file is loaded:

- The VM assumes full ownership of the bytecode buffer
- The assembler has no further role
- The bytecode buffer is treated as read-only

13.7 Validation Before Execution

Before execution begins, the VM validates the input bytecode:

- Every opcode is checked for validity
- Instructions requiring operands are checked for truncation
- The presence of a HALT instruction is verified

If validation fails, execution does not begin.

13.8 Error Handling for Invalid Input

If the supplied input is invalid, the VM must:

- Print an error message to `stderr`
- Terminate execution immediately
- Exit with a non-zero exit code

13.9 Examples

13.9.1 Valid Invocation

```
./vm program.bin
```

Behavior:

- Bytecode is loaded
- Validation succeeds
- Execution may proceed

13.9.2 Invalid Invocation

```
./vm  
./vm a.bin b.bin
```

Behavior:

- Usage message is printed
- VM exits without execution

13.10 What the VM Never Accepts as Input

The VM explicitly rejects:

- Assembly source files (`.asm`)
- Textual input of any kind
- Partially written or truncated bytecode files

13.11 Interface Boundary Summary

Assembler input is text. VM input is binary. The boundary is absolute.

14 Instructions to Build and Run the System

14.1 Prerequisites

The following tools must be installed on the system:

- GCC compiler (C11 compatible)
- GNU Make
- Unix-like operating system (Linux recommended)

14.2 Project Directory Structure

The project is organized as follows:

```
project/  
  assembler/  
    assembler.c  
    assembler.h  
  VM/  
    vm.c  
    vm.h  
  test/  
    test.asm  
  main.c  
  Makefile
```

14.3 Build Instructions

To compile both the assembler and the virtual machine, run:

```
make
```

This produces two executables:

- **assembler** — the assembler binary
- **vm** — the virtual machine binary

To clean all generated binaries, run:

```
make clean
```

14.4 Assembling an Assembly Program

To convert an assembly source file into bytecode, run:

```
./assembler input.asm output.bin
```

Where:

- `input.asm` is a valid assembly program
- `output.bin` is the generated bytecode file

14.5 Running the Virtual Machine

To execute or validate the bytecode file using the VM, run:

```
./vm output.bin
```

The VM performs the following steps:

1. Loads the bytecode file into memory
2. Validates bytecode structure
3. Optionally dumps bytecode for inspection
4. Executes the program (if execution is enabled)

14.6 Example End-to-End Execution

Given an assembly file `test/test.asm`:

```
PUSH 10  
HALT
```

The complete workflow is:

```
make  
./assembler test/test.asm test/test.bin  
./vm test/test.bin
```

Expected bytecode output (hexadecimal):

```
01 0A 00 00 00 FF
```

14.7 Error Handling During Execution

If incorrect usage or invalid input is detected, the system behaves as follows:

- Incorrect CLI usage prints a usage message
- Missing or unreadable files produce an error message
- Invalid or truncated bytecode halts execution immediately

All errors are reported via `stderr`, and the program exits with a non-zero status code.

14.8 Execution Boundary Clarification

The assembler is run once to produce bytecode. The VM is run separately to consume that bytecode. No component performs both roles.